

JavaScript

Chapitre 05

Requêtes HTTP

Le protocole HTTP

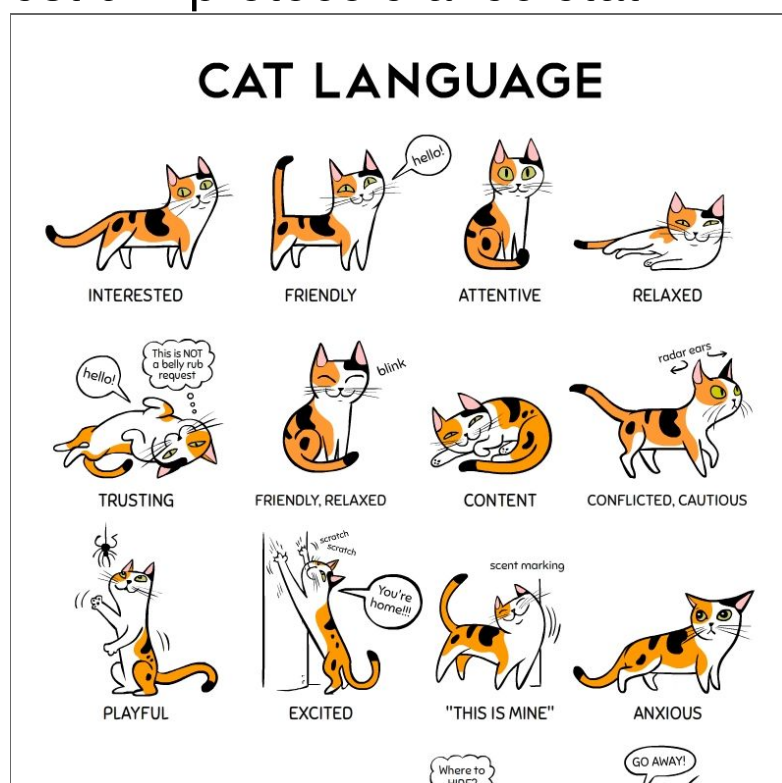
Requêtes HTTP

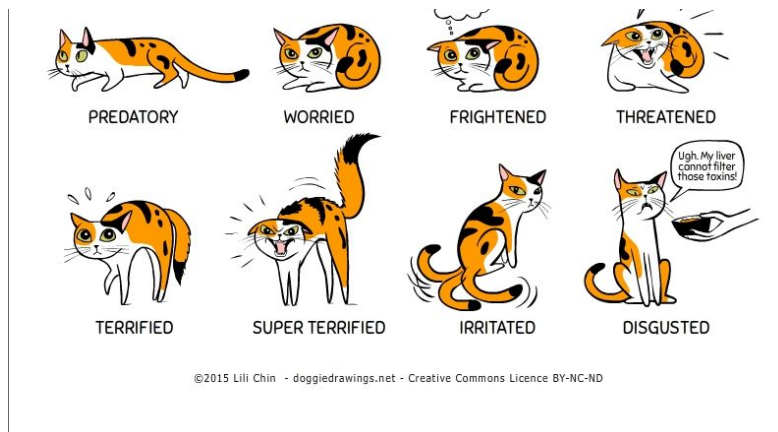
Le protocole HTTP

Le protocole HTTP (Hypertext Transfer Protocol) est le protocole qui est à la base de la communication entre le navigateur et le serveur web (Apache / nginx, etc...). Ce protocole formalise les modalités de transmission de documents hypermédias (HTML, image, son, etc.). Le protocole HTTP est un protocole dit "protocole sans état".

Un protocole sans état est un protocole de communication qui n'enregistre pas son état entre deux requêtes successives. Les requêtes sont indépendantes les unes des autres.

Dans le cas d'un "protocole **avec** état" (stateful protocol), le serveur conserve l'ensemble des états entre chaque requête. Par exemple, le protocole FTP est un "protocole avec état".





Cat language

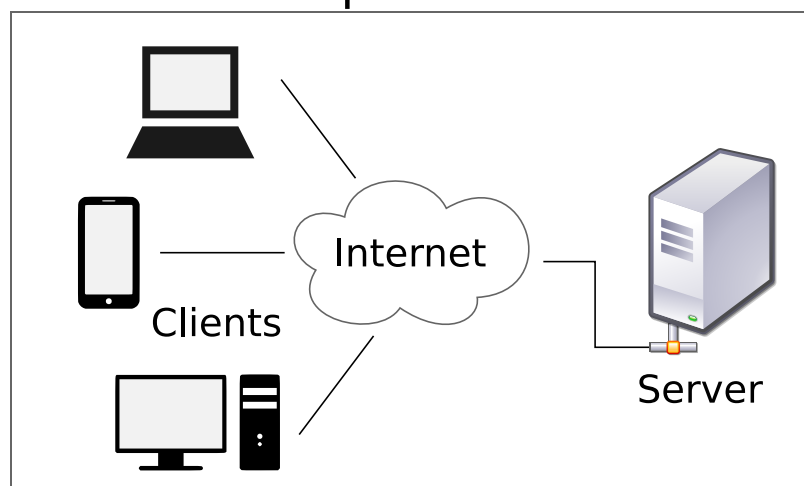
source: <https://www.hauspanther.com/2016/12/04/cat-language-illustrations-lili-chin/>

Principe

Le protocole HTTP suit le model client-serveur :

D'un côté on a un fournisseur (le serveur) et de l'autre des demandeurs (les clients).

- . Le client, initialise une connexion avec le serveur ;
- ?. Une fois la connexion établie, le client envoie une demande : c'est la **requête** ;
- }. Le serveur envoie les données demandées en retour : c'est ce que l'on nomme la **réponse** ;
- !. La connexion prend fin.



Modèle client-serveur

source: https://en.wikipedia.org/wiki/Client%E2%80%93server_model

requête HTTP

Structure

Une requête HTTP est composée des éléments suivants :

- Une ligne de requête qui contient la méthode HTTP utilisée par le client et la version du protocole considérée ;
- Un ensemble (facultatif) d'en-têtes HTTP ;
- Une ligne vide ;
- Le corps de la requête (facultatif) qui contient les données associées au message HTTP.

Exemple

Contenu de la requête permettant d'obtenir la page (ressource) :

http://iut.univ-amu.fr/index.php

```
GET /index.php HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64;
x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/105.0.0.0 Safari/537.36
Host: iut.univ-amu.fr
```

```
user1@Galia:~$ telnet iut.univ-amu.fr 80
Trying 139.124.244.54...
Connected to betelgeuse2.pj.univ-amu.fr.
Escape character is '^]'.
GET /index.php HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KH
Host: iut.univ-amu.fr
```


Les méthodes HTTP

HTTP définit un ensemble de méthodes de requête (verbes HTTP) qui permettent d'indiquer l'action que l'on souhaite exécuter sur la ressource.

Propriétés

Ces méthodes sont dites :

- **sécurisée** (*Safe* en US) :

Si elle ne modifie pas l'état du serveur c-à-d que l'action est en lecture seule.

- **Idempotente** (*Idempotent* en US) :

Si une requête identique peut être faite une ou plusieurs fois de suite avec le même effet, tout en laissant le serveur dans le même état.

Liste des méthodes HTTP

source : <https://developer.mozilla.org/fr/docs/Web/HTTP/Methods>, <https://www.rfc-editor.org/rfc/rfc7231#section-4.2>

Méthode	Action	Safe	<i>idempotent</i>
GET	Demande une représentation de la ressource spécifiée. Les requêtes GET doivent uniquement être utilisées afin de récupérer des données.	✓	✓
HEAD	Demande une réponse identique à une requête GET pour laquelle on aura omis le corps de la réponse (on a uniquement l'en-tête).	✓	✓
POST	Est utilisée pour envoyer une entité vers la ressource indiquée. Cela entraîne généralement un	✗	✗

Méthode	Action	Safe	<i>idempotent</i>
	changement d'état ou des effets de bord sur le serveur.		
PUT	Remplace toutes les représentations actuelles de la ressource visée par le contenu de la requête.	✗	✓
DELETE	Supprime la ressource indiquée.	✗	✓
CONNECT	Établit un tunnel vers le serveur identifié par la ressource cible.	✗	✗
OPTIONS	Est utilisée pour décrire les options de communications avec la ressource visée.	✓	✓
TRACE	Réalise un message de test aller/retour en suivant le chemin de la ressource visée.	✓	✓
PATCH	Est utilisée pour appliquer des modifications	✗	✗

Méthode	Action	Safe	<i>idempotent</i>
	partielles à une ressource.		

Réponse HTTP

Structure

Le contenu d'une réponse HTTP est composée de :

- Une ligne de statut qui indique la version du protocole, le code de statut et sa signification ;
- Un ensemble de champs d'entête (facultatif) donnant des informations sur la réponse et le serveur ;
- Le corps de la réponse, contenant les données.

Exemple

```
user1@Galia:~$ telnet iut.univ-amu.fr 80
Trying 139.124.244.54...
Connected to betelgeuse2.pj.univ-amu.fr.
Escape character is '^]'.
GET /index.php HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KH
Host: iut.univ-amu.fr
```

```
HTTP/1.1 301 Moved Permanently
Server: nginx
Date: Fri, 23 Sep 2022 10:00:44 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive
Location: https://iut.univ-amu.fr/index.php
```

```
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>
Connection closed by foreign host.
```

```
HTTP/1.1 301 Moved Permanently
Server: nginx
Date: Fri, 23 Sep 2022 10:00:44 GMT
Content-Type: text/html
```

```
Content-Length: 162
Connection: keep-alive
Location: https://iut.univ-amu.fr/index.php
```

```
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

Structure de la réponse

Statut

Le protocole HTTP définit une liste de codes (codes HTTP) qui permettent d'obtenir des informations concernant l'état de la requête. Ces codes sont présents dans le contenu de la réponse du serveur et permettent aux applications qui consomment la requête de réagir à son état.

```
user1@Galia:~$ telnet iut.univ-amu.fr 80
Trying 139.124.244.54...
Connected to betelgeuse2.pj.univ-amu.fr.
Escape character is '^]'.
GET /index.php HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/105.0.0.0 Safari/537.36
Host: iut.univ-amu.fr

HTTP/1.1 301 Moved Permanently
Server: nginx
Date: Fri, 23 Sep 2022 10:00:44 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive
Location: https://iut.univ-amu.fr/index.php

<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>
Connection closed by foreign host.
```

Exemple du résultat d'une même requête réalisée via telnet et dans un navigateur web

Principaux codes HTTP

La liste complète des codes HTTP est disponible sur Wikipédia.

source : [https://fr.wikipedia.org/wiki/](https://fr.wikipedia.org/wiki/Liste_des_codes_HTTP)

Liste des codes HTTP

2xx - Succès

Code	Message	Signification
200	OK	Requête traitée avec succès. La réponse dépendra de la méthode de requête utilisée.
201	Created	Requête traitée avec succès et création d'un document.
202	Accepted	Requête traitée, mais sans garantie de résultat.
203	Non-Authoritative Information	Information retournée, mais générée par une source non certifiée.
204	No Content	Requête traitée avec succès mais pas d'information à renvoyer.

3xx - Redirection

Code	Message	Signification
------	---------	---------------

Code	Message	Signification
301	Moved Permanently	Document déplacé de façon permanente.
304	Not Modified	Document non modifié depuis la dernière requête.

4xx - Erreur du client HTTP

Code	Message	Signification
400	Bad Request	La syntaxe de la requête est erronée.
401	Unauthorized	Une authentification est nécessaire pour accéder à la ressource.
402	Payment Required	Paielement requis pour accéder à la ressource.
403	Forbidden	Le serveur a compris la requête, mais refuse de l'exécuter. Contrairement à l'erreur 401, s'authentifier ne fera aucune différence. Sur les serveurs où l'authentification est requise, cela signifie généralement que l'authentification a été acceptée mais que les droits d'accès ne permettent pas au client d'accéder à la ressource.
404	Not Found	Ressource non trouvée.
418	I'm a teapot	« Je suis une théière » : Ce code est défini dans la RFC 2324 datée du 1er avril

Code	Message	Signification
		1998, Hyper Text Coffee Pot Control Protocol.

5xx - Erreur du serveur

Code	Message	Signification
500	Internal Server Error	Erreur interne du serveur.
503	Service Unavailable	Service temporairement indisponible ou en maintenance.
504	Gateway Time-out	Temps d'attente d'une réponse d'un serveur à un serveur intermédiaire écoulé.

Les entêtes HTTP

Les entêtes HTTP sont généralement classées selon 4 catégories :

- . General Headers
- . Request Headers
- . Response Headers
- . Representation headers.

```
POST / HTTP/1.1
Host: localhost:8000
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 6_8; rv:51.0) Gecko/20100101 Firefox/51.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data; boundary=-12656974
Content-Length: 345

-12656974
(more data)
```

Request headers

General headers

Representation headers

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Wed, 10 Aug 2016 13:17:18 GMT
Etag: "d9b3b803e9a0dc6f22e2f20a3e90f69c41f6b71b"
Keep-Alive: timeout=5, max=999
Last-Modified: Wed, 10 Aug 2016 05:38:31 GMT
Server: Apache
Set-Cookie: csrf_token=...
Transfer-Encoding: chunked
Vary: Cookie, Accept-Encoding
X-Frame-Options: DENY

(body)
```

Response headers

Representation headers

General headers

Structure de l'entête d'une requête

source : [https://
developer.mozilla.org/en-
US/docs/Web/HTTP/
Messages](https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages)

Structure de l'entête d'une réponse

source : [https://
developer.mozilla.org/en-
US/docs/Web/HTTP/
Messages](https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages)

General Headers

Les Entêtes générales (on ne fait plus nécessairement cette distinction aujourd'hui) regroupent les entêtes qui ne donne pas d'information particulières concernant la requête, la réponse ou le message lui même (la requête). En générale, elles donnent des informations sur la façon dont la requête va être traitée.

Request Headers (requête uniquement)

Les entêtes de requête donne des informations sur le contexte de la requête. Cela permet au serveur d'adapter sa réponse. Par exemple, on retrouve dans cet entête l'entête **Authorization** qui permet de fournir des information de connexion au serveur.

```
Authorization: Basic YWxhZGRpbjpvcGVuc2VzYW1l
```

Response Headers (réponse uniquement)

Les entêtes de réponse donnent des informations sur le contexte de la réponse. On y retrouve par exemple l'entête **Location** qui indique l'url qu'il faut suivre pour retrouver une page qui a été déplacée.

Representation headers

Les entêtes de représentation donnent des informations sur la représentation des données fournies dans le corps du message HTTP. Par exemple, pour

une requête on va y retrouver l'entête **Content-Type: application/x-www-form-urlencoded** qui indique comment sont formaté les données fournies dans le corps de la requête. Et, pour une réponse, on peut y retrouver des **Content-Type: text/html** qui indique que les données du corps correspondant à du texte au format HTML.

Identifier une ressource sur le web

URI

Pour identifier une ressource sur le web, on utilise une **URI** (Uniform Resource Identifier). Il s'agit d'une chaîne de caractères qui identifie de manière unique une ressource sur internet par son nom, sa localisation, voir les deux.

URL

Une **URL** (Uniform Resource Locator) est une URI particulière qui identifie une ressource sur internet par sa localisation. Ainsi, on sait exactement où la trouver. Une URL spécifie aussi le moyen d'y accéder (*http*, *ftp*, *mailto*, etc.).

URN

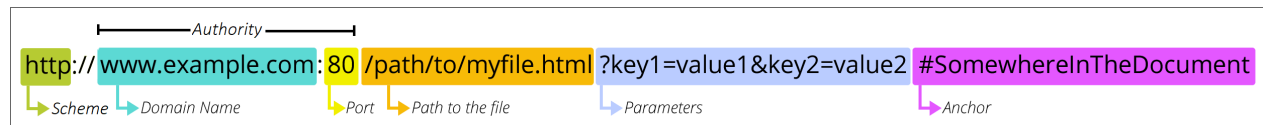
Une **URN** (Uniform Resource Name) quand à elle est une URI qui identifie une ressource par son nom. Celle-ci est donc identifiée indépendamment de sa localisation. Dans ce cas il est nécessaire de posséder un mécanisme permettant de savoir où elle se situe. En général on utilise un dictionnaire de correspondance entre URN et URL. Une URN est définie par un schéma de nommage spécifique (cf. https://fr.wikipedia.org/wiki/Uniform_Resource_Name).

aller plus loin : **<https://danielmiessler.com/study/difference-between-uri-url/>**

Qu'est-ce qu'une URL

Taxonomie d'une URL

source : https://developer.mozilla.org/en-US/docs/Learn/Common_questions/What_is_a_URL



Description

élément	description
http://	Indique quel protocole doit être utilisé (<i>http</i> , <i>ftp</i> , <i>mailto</i> , etc.).
www.example.com	Indique le nom de domaine. Cela sert à identifier le serveur cible de la requête. On peut aussi indiquer une adresse IP.
:80	Port utilisé par le serveur web. Le port par default est le port 80 . Dans ce cas, il est inutile de le

élément	description
	préciser.
/path/to/myfile.html	Chemin d'accès de la ressource sur le serveur web.
?key1=value1&key2=value2	Paramètres supplémentaires fournis au serveur web sous la forme de paires clé/valeur.
#SomewhereInTheDocument	Ancre vers un endroit identifié de la page web.

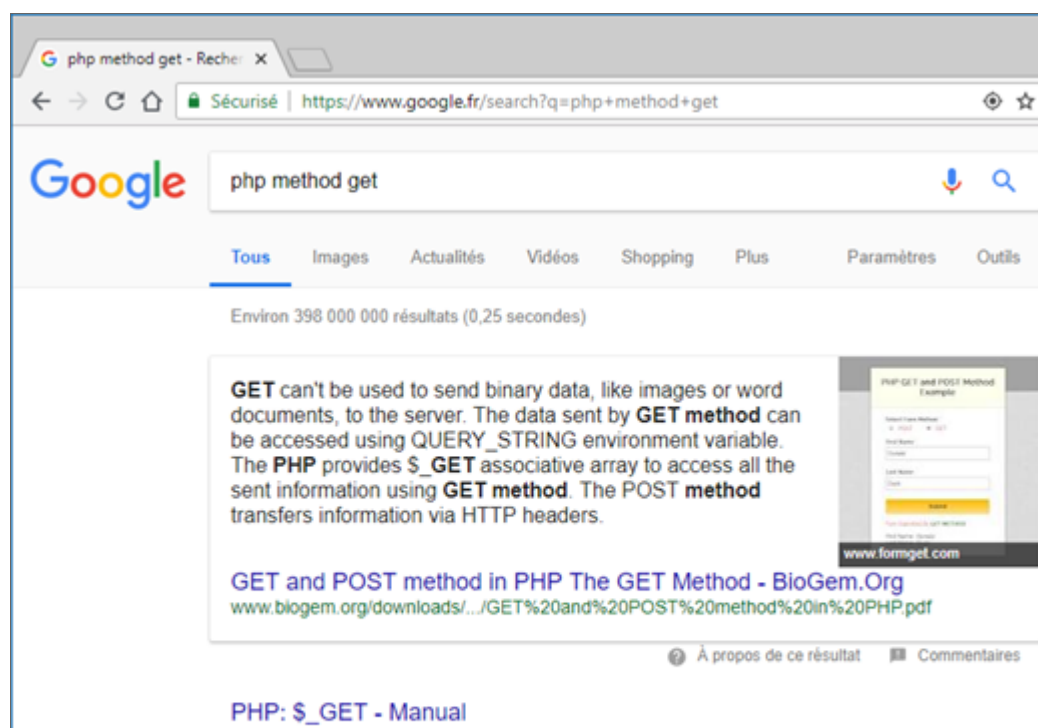
Envoie de données via l'URL

Lorsque l'on envoie une requête au serveur web, il est possible de lui transmettre des paramètres supplémentaires. Ces paramètres peuvent par exemple être utilisés par le serveur pour produire ou définir la ressource qui va être envoyée.

Lorsque l'on fait une recherche sur google, la page est rechargée avec le résultat de la requête. Le serveur a donc bien reçu les paramètres de la recherche "php method get". Mais, comment ?

C'est la capacité à ajouter des paramètres dans l'URI du protocole HTTP qui a été utilisée ici.

Les paramètres sont récupérés côté serveur à partir de la requête et sont utilisés lors de la génération de la réponse.



[php.net/manual/en/reserved.variables.get.php](#) ▼ Traduire cette page

The variable name `$_GET` is a bit misleading. It works with any HTTP request method that has a query component in the URI: GET, POST, PUT, PATCH, ...

PHP 5 Form Handling - W3Schools

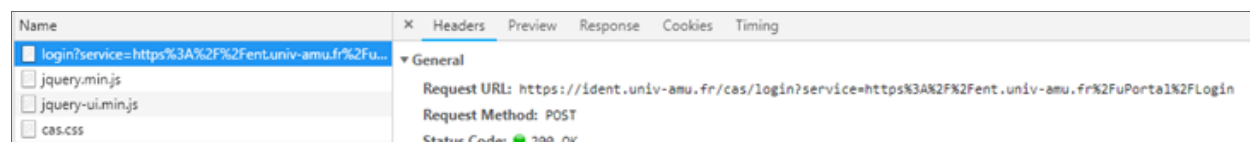
https://www.w3schools.com/php/php_forms.asp ▼ Traduire cette page

Envoie de données dans le corps du message HTTP

Le protocole HTTP prévoit l'envoi de données via les méthodes POST et PUT. Les données sont passées directement dans le corps de la requête. Elles peuvent ensuite être récupérées par le serveur.



Lors de l'envoi du formulaire de connexion, le login et le mot de passe ne sont pas visibles dans l'URI... OUF ! Ces données sont envoyées dans l'entête HTTP, visible ci-dessous... Outch !



The screenshot displays the 'Form Data' section of a web browser's developer tools, specifically the 'Network' tab. The left sidebar shows a list of resources, with 'wrong4_red.png' selected. The main panel shows the details of the selected resource, which is a login form. The 'Form Data' section is highlighted with a red box and contains the following information:

- username:** desbenoit
- password:** mon mot de passe
- lt:** LT-3792701-ACjdPRDsSIUjTKuI10FHKi@Qcdc6CI-ident.univ-amu.fr
- execution:** e1s2
- _eventId:** submit
- submit:** SE CONNECTER

Other visible details include the 'Remote Address' (139.124.244.65:443), 'Referer Policy' (no-referrer-when-downgrade), and 'Query String Parameters' (service: https://ent.univ-amu.fr/uPortal/Login).

Formatage des données

Le format d'échange des données est indiqué par le champ **Content-type** (cf. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST>).

Content-type	Description
application/x-www-form-urlencoded	Valeurs encodées sous la forme de paires clé/valeur, comme pour l'URL. Les caractères non alphanumériques seront encodé avec des %.
multipart/form-data	Chaque valeur est envoyée sous la forme de blocs de données, séparée par des frontières définies par l'agent utilisateur.
text/plain	Les valeurs sont envoyées au format text.

Exemple

Message HTTP contenant des données dans le corps.

```
POST /path/script.cgi HTTP/1.0
From: frog@jmarshall.com
User-Agent: HTTPTool/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 32
```

```
home=Cosby&favorite+flavor=flies
```

source <http://www.jmarshall.com/easy/http/>

```
POST /test HTTP/1.1
Host: foo.example
Content-Type: multipart/form-
data;boundary="boundary"

--boundary
Content-Disposition: form-data; name="field1"

value1
--boundary
Content-Disposition: form-data; name="field2";
filename="example.txt"

value2
--boundary--
```

source [https://developer.mozilla.org/en-US/docs/](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST)
[Web/HTTP/Methods/POST](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST)

AJAX

Requêtes HTTP

Ajax, qu'est-ce que c'est ?

AJAX n'est pas un nouveau langage ! Sous ce terme se cache simplement l'utilisation conjointe de plusieurs technologies : HTML/XHTML, CSS, JavaScript, XML

Qu'entend-on par AJAX ?

AJAX : Asynchronous Javascript And XML

JavaScript : le langage

asynchrone : le mode opératoire

xml : le format d'échange

Pourquoi faire ?

L'objectif consiste à combiner l'ensemble de ces technologies pour permettre aux applications web de mettre à jour rapidement leurs contenus, et ce, sans rechargement de la page. Cela permet de concevoir des applications plus reactive.

Aujourd'hui, dans la majorité des cas, les applications utilisent le format JSON pour réaliser l'échange de données et non pas le XML comme le laisserait supposer le terme **AJAX**.

Architectures d'une application AJAX

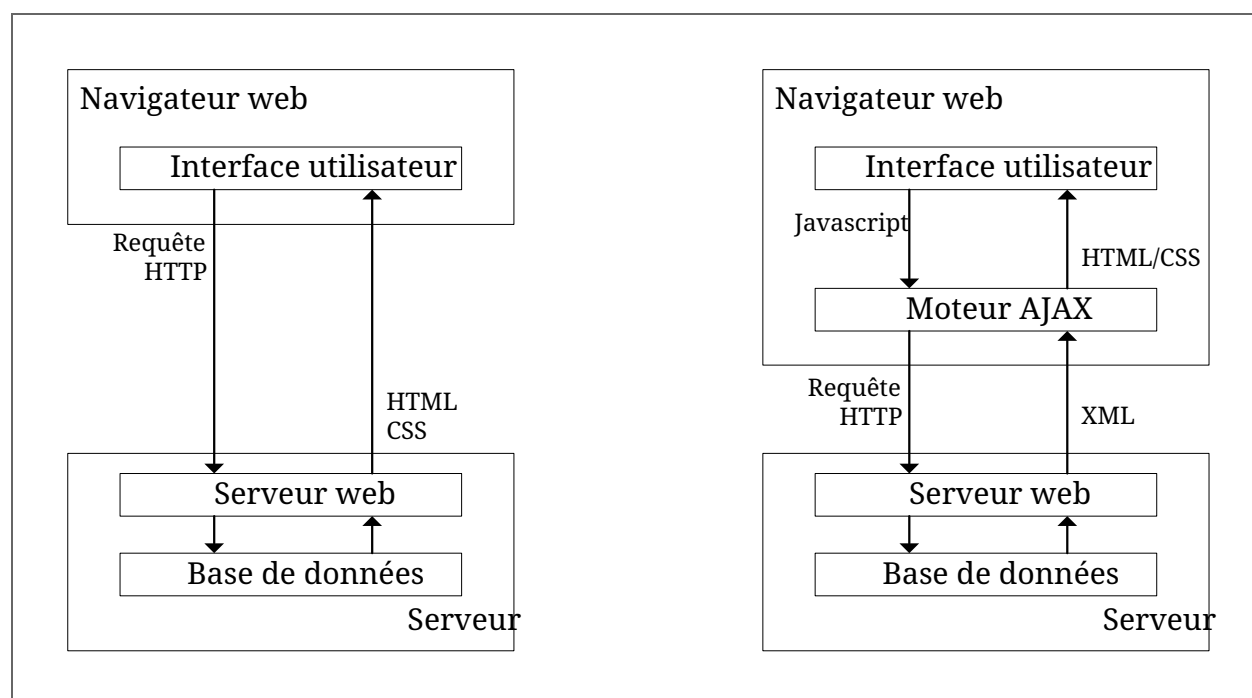
A la différence d'une application web classique, une application AJAX va tirer partie de la capacité à faire des requêtes côté client (dans le navigateur).

Le plus souvent, le résultat des requêtes est exploité pour modifier le DOM de la page.

Cela permet d'avoir une page dynamique sans rechargement améliorant ainsi l'expérience de l'utilisateur (UX).

Deux questions se posent alors :

- . Quel type de données peut-on échanger/charger ?
- ?. Comment ?



Format d'échange de données

Requêtes HTTP

Les différents formats d'échange

Lors de l'utilisation de requêtes HTTP côté client, les trois formats d'échange ci-dessous sont les plus utilisés avec toutefois une large prédominance du premier.

JSON

Permet de manipuler directement un objet javascript.

XML

Permet de manipuler directement une arborescence avec les mêmes méthodes que le DOM.

texte

Permet de retourner directement des données brutes comme par exemple du "texte html".

Le format XML

Le langage **XML** (eXtensible Markup Language) est un langage de description basé sur la notion de balisage. C'est un format décrit et structure l'information. L'un de ces grand avantage c'est qu'il est aisément lisible par l'homme. En revanche, il peut être très *verbeux*. C'est un format qui permet de faciliter l'échange de données entre systèmes d'informations.

Validation

Un document XML est définissable et validable par un DTD (Document type definition).

Aller plus loin

[https://fr.wikipedia.org/wiki/](https://fr.wikipedia.org/wiki/Document_type_definition)

[Document type definition](https://fr.wikipedia.org/wiki/XML_Schema)

https://fr.wikipedia.org/wiki/XML_Schema

Transformation

Un document XML est entièrement transformable en un autre document XML. On peut par exemple utiliser une feuille XSLT (Extensible Stylesheet Language Transformations) pour cela.

Aller plus loin

[https://fr.wikipedia.org/wiki/](https://fr.wikipedia.org/wiki/Extensible_Stylesheet_Language_Transformations)

[Extensible Stylesheet Language Transformations](https://fr.wikipedia.org/wiki/Extensible_Stylesheet_Language_Transformations)

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<personnes>
```

```
<personne>
  <nom>Dupond</nom>
  <service>Achats</service>
</personne>
<personne>
  <nom>Durand</nom>
  <service>Achats</service>
</personne>
<personne>
  <nom>Dupuis</nom>
  <service>Courrier</service>
</personne>
</personnes>
```

Exemple de document XML

Le format JSON

La notation JSON (JavaScript Object Notation) est un format standard utilisé pour représenter des données structurées de façon semblable aux objets Javascript. Sa spécification complète est disponible sur le site web <https://www.json.org>.

Éléments de base du langage

Les objets

Ensemble de couples nom/valeur séparés par des virgules et entre accolades.

```
{ "nom":valeur, "nom":valeur, ...}
```

Les tableaux

Ensemble de valeurs séparées par des virgules.

```
[valeur, valeur, valeur, ...]
```

Les valeurs

Peuvent être, soit une chaîne de caractères entre guillemets, soit un nombre, soit **true** ou **false** ou **null**, soit un objet soit un tableau.

De JSON à Javascript

Un JSON est une chaîne de caractères. Il ne s'agit pas d'un littéral objet et doit donc être converti pour pouvoir être utilisé. Pour cela, il faut en faire un objet Javascript.

Comment passer d'un JSON à un objet Javascript ?

Pour transformer une chaîne de caractères représentant un JSON à un objet Javascript on peut utiliser au choix la fonction `eval`, ou la méthode `parse` de l'objet Javascript `JSON`. La première solution est fortement déconseillée pour des raisons de sécurité. Il est préférable d'utiliser `JSON.parse(text [, reviver])`.

Exemple

Conversion d'un objet JSON stocké dans une chaîne de caractères en un objet Javascript.

```
var json = '{"result":true, "count":42}'; // json
: Notre JSON
const obj = JSON.parse(json);           // obj
: objet Javascript correspondant à notre JSON
console.log(obj.count); //42             // On
peut maintenant utiliser obj comme un objet
Javascript classique
```


Manipuler des données JSON

superheroes.json

```
{
  "squadName": "Super hero squad",
  "homeTown": "Metro City",
  "formed": 2016,
  "secretBase": "Super tower",
  "active": true,
  "members": [
    {
      "name": "Molecule Man",
      "age": 29,
      "secretIdentity": "Dan Jukes",
      "powers": [
        "Radiation resistance",
        "Turning tiny",
        "Radiation blast"
      ]
    },
    {
      "name": "Eternal Flame",
      "age": 1000000,
      "secretIdentity": "Unknown",
      "powers": [
        "Immortality",
        "Heat Immunity",
        "Inferno",
        "Teleportation",
        "Interdimensional travel"
      ]
    }
  ]
}
```

L'exemple ci-contre est tiré de la documentation MDN

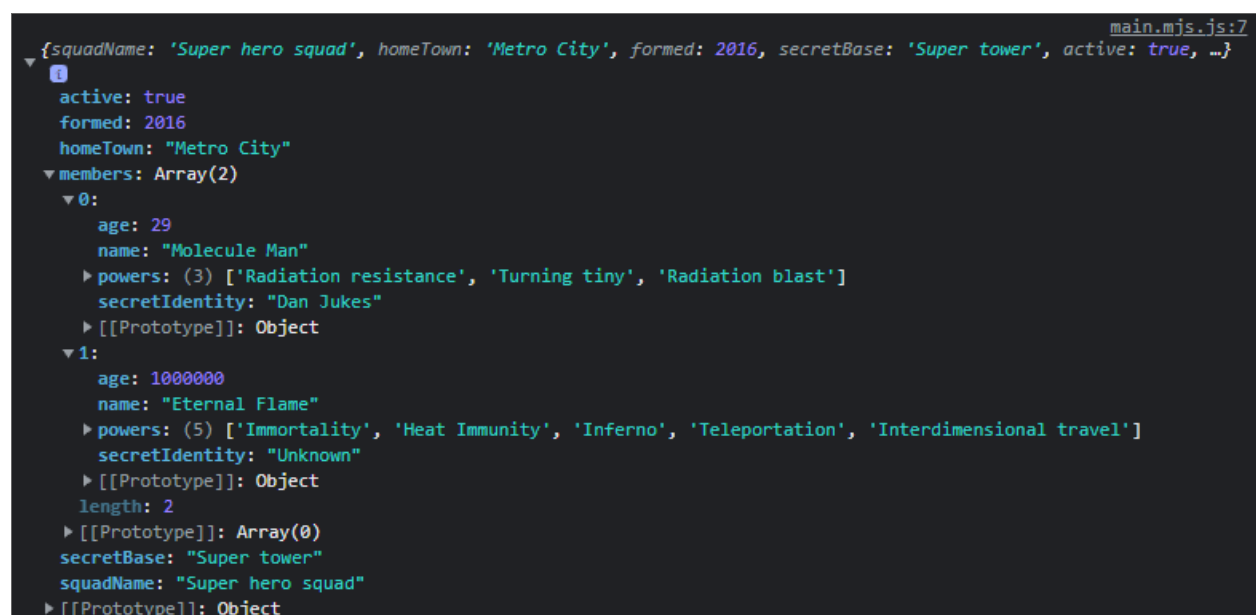
Source : <https://developer.mozilla.org/fr/docs/Learn/>

JavaScript/Objects/JSON

Pour pouvoir utiliser en Javascript un objet JSON, il faut deux choses :

- . Convertir l'objet JSON en objet Javascript ;
- ?. Connaître la structure du JSON à notre disposition.

Si l'on n'est pas à l'origine de la création du JSON qu'on utilise (ex. service web), on peut soit se référer à la documentation si elle existe, soit afficher l'objet converti dans la console développeur afin de l'étudier.



Chargement (cf. [v8-features](#)) et utilisation de l'objet ci-contre :

```
import superHeroes from './superheroes.json' assert
{type: 'json'} // Oui, on peut !
console.log(superHeroes) // { "squadName": "Super
hero squad", "homeTown": "Metro City",...

console.log(superHeroes.homeTown)
// Metro City
console.log(superHeroes['active'])
// true
console.log(superHeroes['members'][1]['powers'][2])
```

// Inferno

L'objet `XMLHttpRequest`

Requêtes HTTP

Création d'un objet XMLHttpRequest

Pour réaliser des requêtes HTTP via un script dans la page web, nous allons utiliser l'objet **XMLHttpRequest**.

Historiquement c'est Microsoft qui a introduit l'objet **XMLHttpRequest** sous la forme d'un composant ActiveX. Depuis 2012 (travail de normalisation de 2006 à 2011), l'objet **XMLHttpRequest** fait partie de la norme sous la forme d'un objet Javascript. Cet objet définit une API (interface d'application) qui permet au script client d'échanger des données avec un serveur distant.

La création d'un objet **XMLHttpRequest** se fait simplement à l'aide du constructeur de la classe.

```
const xhr = new XMLHttpRequest();
```

Une fois l'objet construit, il faut encore initialiser la requête, puis l'envoyer. Pour ça on va utiliser les méthodes **open** et **send**.

C'est bon à savoir

- . La norme prévoit la possibilité de faire des requêtes de même origine (seule possibilité historiquement), mais aussi des requêtes d'origines différentes.
- ?. Le nom court de XMLHttpRequest est **XHR**.

Initialisation de la requête

La méthode **open** de l'objet XMLHttpRequest permet d'initialiser une nouvelle requête (ou re-initialiser une requête existante).

Syntaxe

```
open(method, url)
open(method, url, async)
open(method, url, async, user)
open(method, url, async, user,
password)
```

Paramètres

method

La méthode HTTP à utiliser (GET, POST, ...).

url

l'url à laquelle envoyer la requête

async

Un booléen pour indiquer si elle doit être asynchrone (true) ou synchrone (false).

user, password

Paramètres de connexion.

Synchrone ou asynchrone

La requête HTTP peut être exécutée de manière synchrone ou asynchrone (cas par défaut).

Synchrone

Le script va attendre le résultat du retour de la requête pour poursuivre son exécution.

Attention, le mode synchrone est aujourd'hui déprécié à cause de l'impact qu'il peut avoir sur l'expérience utilisateur.

Asynchrone

Le script n'attend pas, il faut passer par les gestionnaires d'événement pour traiter le résultat de la requête quand il sera disponible.

Envoie de la requête

L'envoi de la requête se fait via la méthode **send** de l'objet XMLHttpRequest.

```
const xhr = new XMLHttpRequest();
xhr.open('GET', '/server', true);

// ...
xhr.send();
```

Cette méthode peut également prendre un paramètre **body**. Ce paramètre permet de définir le contenu du corps de la requête. Il est utilisé pour transmettre des données.

Dans ce cas, attention à bien utiliser la méthode HTTP POST et non pas GET. Il faut également veiller à définir le format utilisé pour représenter les données. On peut par exemple utiliser la méthode **setRequestHeader** pour ça (cf. <https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/send>) :

```
const xhr = new XMLHttpRequest();
xhr.open("POST", '/server', true);

//Send the proper header information along with the
request
xhr.setRequestHeader("Content-Type", "application/
x-www-form-urlencoded");

// ...
```

```
xhr.send("foo=bar&lorem=ipsum");
```

Statut de la requête

L'objet XHR possède une propriété **readyState** qui permet de connaître l'état de la requête.

Value	State	Description
0	UNSENT	Le client a été créé. open() n'a pas encore été appelé.
1	OPENED	open() a été appelé.
2	HEADERS_RECEIVED	send() a été appelé, et les en-têtes et le statut sont disponibles.
3	LOADING	Téléchargement; responseText contient des données partielles.
4	DONE	L'opération est terminée.

A chaque changement de cet état, l'événement **readystatechange** est lancé. Il est possible d'y attacher un gestionnaire d'événement pour suivre l'évolution de la requête et déclencher des actions en fonction de l'état.

XHR: l'événement `readystatechange`

Pour attacher un gestionnaire d'événement à l'événement `readystatechange`, on peut au choix, utiliser attacher le gestionnaire avec la fonction `addEventListener`, ou définir un gestionnaire comme à la propriété `onreadystatechange`.

```
addEventListener('readystatechange', (event) => {  
  })  
onreadystatechange = (event) => { }
```

Dans l'extrait de code ci-dessous, on crée un objet `xhr` et on attache à l'événement `readystatechange` un gestionnaire qui permet de suivre l'état de la requêtes.

```
const xhr = new XMLHttpRequest();  
const method = "GET";  
const url = "https://developer.mozilla.org/";  
  
xhr.open(method, url, true);  
xhr.onreadystatechange = () => {  
  // In local files, status is 0 upon success in  
  Mozilla Firefox  
  if (xhr.readyState === XMLHttpRequest.DONE) {  
    const status = xhr.status;  
    if (status === 0 || (status >= 200 && status <  
400)) {  
      // The request has been completed  
      successfully  
      console.log(xhr.responseText);  
    } else {  
      // Oh no! There has been an error with the
```

```
request!  
    }  
}  
};  
xhr.send();
```

source https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/readystatechange_event

De XHR à XHR2

Avec l'arrivée de la norme XHR2, plusieurs événements ont été ajoutés tels que **onload**, **onprogress**, ou encore **onerror**. Les développements modernes s'appuient donc davantage sur ces événements que sur **onreadystatechange**. Cela permet de rendre le code plus lisible.

```
const xhr = new XMLHttpRequest();
console.log('UNSENT', xhr.readyState); //
readyState will be 0

xhr.open('GET', '/api', true);
console.log('OPENED', xhr.readyState); //
readyState will be 1

xhr.onprogress = () => {
  console.log('LOADING', xhr.readyState); //
  readyState will be 3
};

xhr.onload = () => {
  console.log('DONE', xhr.readyState); //
  readyState will be 4
};

xhr.send(null);
```

source <https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/readyState>

Type de réponse

Pour récupérer le résultat de la requête, on peut passer par l'attribut **reponseText** ou **reponseXML** pour avoir respectivement la réponse sous forme de texte ou sous la forme d'un document XML.

Il est aussi possible d'utiliser l'attribut **reponseType** pour spécifier le type de réponse attendu. L'attribut **reponse** contiendra alors la valeur de retour au format spécifié par ce dernier.

```
const xhr = new
XMLHttpRequest();
xhr.open('GET', '/
server', true);

// If specified,
responseType must be
empty string or "text"
xhr.responseType =
'text';

xhr.onload = () => {
  if (xhr.readyState
=== xhr.DONE) {
    if (xhr.status ===
200) {

console.log(xhr.respons
e);

console.log(xhr.respons
eText);
    }
  }
};

xhr.send(null);
```

```
const xhr = new
XMLHttpRequest;
xhr.open('GET', '/
server');

// If specified,
responseType must be
empty string or
"document"
xhr.responseType =
'document';

// Force the response
to be parsed as XML
xhr.overrideMimeType('t
ext/xml');

xhr.onload = () => {
  if (xhr.readyState
=== xhr.DONE &&
xhr.status === 200) {

console.log(xhr.respons
e, xhr.responseXML);
  }
};

xhr.send();
```

<https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/responseText>, **<https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/responseXML>**

Exemple

Cas d'une requête HTTP traitée de manière asynchrone avec des paramètres passés dans l'URL (GET)

```
const sort='name'
const rows=10
const url = `https://opendata.paris.fr/api/
records/1.0/search/?dataset=velib-disponibilite-en-
temps-reel&sort=${sort}&rows=${rows}`
export const xhr = new window.XMLHttpRequest()

xhr.onload = (event) => {
  if (xhr.status === 200) {
    let data = JSON.parse(xhr.responseText)
    console.log(data)
  }
}

xhr.onerror = (event) => {
  console.log(`Une erreur ${event.target.status}
s'est produite au cours de la réception du
document.`)
}

xhr.open('GET', url)
xhr.send(null)
```

Exemple 2

On peut utiliser tout ce qui précède pour coder un composant (attention il est très sommaire) permettant de simplifier l'utilisation de l'objet **XMLHttpRequest** dans nos projets.

xhr.js

```
const XHR = (() => {
  const getJson = ({
    url = '',
    method = 'GET',
    body = null,
    callback = (data) => {
      console.log(data)
    }
  } = {}) => {
    const oReq = new XMLHttpRequest()

    oReq.onprogress = (event) => {
      console.log('LOADING', oReq.readyState);
    }

    oReq.onload = (event) => {
      console.log('DONE', oReq.readyState);
      const data = oReq.response
      callback(data)
    }

    oReq.onerror = (event) => {
      console.log('ERROR', oReq.readyState);
    }
  }
})
```

```
...

oReq.responseType = 'json'
```

```
    oReq.open(method, url)
    oReq.send(body)
  }

  return {
    getJson
  }
})()

export { XHR }
```

Et pour s'en servir :
index.js

```
import { XHR } from './xhr.js'

XHR.getJson({
  url: 'https://api.punkapi.com/v2/beers/random',
  callback: (data) => {
    // On utilise data ici
  }
})
```

L'API Fetch

Requêtes HTTP

L'API Fetch

L'API Fetch (cf. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch) définit une interface pour manipuler les éléments de requêtes HTTP. Elle fournit une définition générique de l'entête (Headers), la requête (Request) et la réponse (Response). Ces interfaces offrent un mécanisme polyvalent pour manipuler ces éléments. Pour effectuer une requête et obtenir une ressource, l'API Fetch fournit une méthode globale nommée **fetch**. Celle-ci permet d'initier le chargement d'une ressource et retourne une promesse qui sera résolue lorsque la réponse sera disponible.

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API :

The Promise returned from `fetch()` won't reject on HTTP error status even if the response is an HTTP 404 or 500. Instead, it will resolve normally (with `ok` status set to `false`), and it will only reject on network failure or if anything prevented the request from completing.

Cette promesse n'échoue pas en cas d'erreur HTTP mais uniquement en cas de problèmes réseaux. De ce fait, une requête faite avec **fetch** ne va pas être traitée exactement comme une requête XHR.

pour aller plus loin :

à propos des fetch - <https://gomakethings.com/why-i-still-use-xhr-instead-of-the-fetch-api/>

concernant les promesses

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

Réaliser une requête avec `fetch`

Pour réaliser une requête avec `fetch`, il suffit de passer en paramètre de la fonction globale `fetch` l'url de la ressource à récupérer. Ensuite, on utilise la méthode `then` pour appeler les gestionnaires attachés à la tenue de la promesse ; d'abords pour résoudre la réponse, puis pour résoudre le contenu du corps de cette dernière *i.e.* les données.

```
fetch('https://api.github.com/repos/axios/axios/commits')
  .then((response) => {
    if (!response.ok) {
      throw new Error(`erreur HTTP! statut: ${response.status}`)
    }
    return response.json();
  })
  .then((data) => {
    // On utilise data ici
  })
```

Remarque :

L'un des gros avantages des promesses ici c'est qu'il devient possible de traiter facilement les dépendances entre requêtes. Par exemple si l'on a 3 requêtes et que la 3e a besoin du résultat des 2 premières (cf. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all).

L'API **Fetch** et les paramètres

Tout comme avec l'objet **XMLHttpRequest**, l'API **Fetch** permet de passer des paramètres dans l'entête et dans le corps de la requête.

```
const options = {
  method: 'POST',

  headers: {
    // Nous n'accepterons que le JSON en résultat.
    'Accept': 'application/json',
    // Dans le cas d'une requête contenant un body,
    // par exemple une POST ou PUT, on définit le
    format du body.
    'Content-Type': 'application/json',
    // Cas d'usage courant pour gérer
    l'authentification avec une API REST.
    'Authorization': 'Bearer ${token}'
  },

  body: JSON.stringify({
    title: 'Un post',
    content: 'Contenu de mon post'
  })
}

fetch('https://example.com/posts', options)
  .then((response) => response.json())
  .then((createdPost) => { /* ... */ })
```

Exemple dans lequel on passe des données dans l'entête et dans le corps de la requête HTTP (source : <https://www.alsacreations.com/tuto/lire/1870-LAPI-Fetch-en-JavaScript-quest-ce-que->

cest-.html)

fetch et await le combo gagnant

Dans la mesure où la méthode globale `fetch` retourne une promesse, il est possible d'utiliser en combinaison avec l'opérateur `await`.

Cet opérateur permet d'attendre la résolution complète d'une promesse.

Attention cependant, il ne peut être utilisé qu'au sein d'une fonction asynchrone (cf. [https://](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function)

developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function) *i.e.* une fonction qui a été déclarée avec le mot clé `async`.

Exemple

Ci-dessous un code écrit avec `async/await` plutôt qu'en utilisant la méthode `then` des promesses.

```
const response = await fetch('https://  
api.punkapi.com/v2/beers/random')  
const data = await response.json()  
// On utilise data ici
```

Exemple

On peut utiliser tout ce qui précède pour coder un composant (attention reste assez sommaire) permettant de simplifier l'utilisation de fetch dans nos projets.

ftc.js

```
const FTC = (() => {
  const getJson = async ({
    url = "",
    method = 'GET',
    body = null
  } = {}) => {
    const response = await fetch(url, {
      method,
      mode: 'cors',
      headers: {
        'Content-Type': 'application/json'
      },
      body
    })
    const result = await response.json()
    return result
  }

  return {
    getJson
  }
})();

export { FTC }
```

index.js

```
import { FTC } from './ftc.js'
```

```
const data = await FTC.getJson({  
  url: 'https://api.punkapi.com/v2/beers/random',  
})  
// On utilise data ici
```

Un mot sur les promesses et `await`

L'utilisation de `await` avec les fonctions asynchrones permet d'écrire du code asynchrone dans un style synchrone. Attention toutefois que cela a un impact sur l'ordre dans lequel votre code va s'exécuter. Lors de l'appel de plusieurs fonctions asynchrones (appel en série), le code va attendre la résolution de chaque fonction asynchrone avant de passer à la suite. De ce fait vous perdrez une grande partie de l'intérêt de travailler avec des appels asynchrones.

Pour s'en rendre compte, vous pouvez étudier le résultat de l'exécution du code ci-dessous.

```
const L = (s, d, t) => (`${' '.repeat(6 -
String(d).length)}${d}${s[1]}${t}${s[2]}`)
const f = async (text, delay, t0, success=true) =>
{
  return new Promise((resolve, reject) => {
    const t1 = performance.now()
    const dt = String(Math.trunc(t1-t0))
    const filled = ' '.repeat(6-dt.length)
    console.log(L`${Math.trunc(t1-t0)}ms -
    ${text}::start`)
    setTimeout((t) => {
      const t2 = performance.now()
      const dt2 = String(Math.trunc(t2-t0))
      const filled = ' '.repeat(6-dt2.length)
      if (success) {
        console.log(L`${Math.trunc(t2-t0)}ms -
        ${t}::resolve`)
        resolve(t)
      } else {
```

```
        console.log(L`${Math.trunc(t2-t0)}ms -  
        ${t}::reject`)  
        reject(t)  
    }  
    }, delay, text)  
  })  
}  
...
```

```
...  
let t0, t1, dt  
let data = []  
t0 = performance.now()  
data.push(await f("f1", 3000, t0))  
data.push(await f("f2", 1000, t0))  
data.push(await f("f3", 4000, t0))  
t1 = performance.now()  
console.log(`${Math.trunc(t1-t0)}ms -`,  
data.join(', '))  
console.log("")  
  
let data2 = []  
t0 = performance.now()  
data2.push(await Promise.all([  
  f("p1", 3000, t0),  
  f("p2", 1000, t0),  
  f("p3", 4000, t0)  
]))  
t1 = performance.now();  
dt = String(Math.trunc(t1-t0))  
console.log(`${Math.trunc(t1-t0)}ms -`,  
data2.join(', '))
```

Un mot sur le debouncing et le throttling

Le debouncing et le throttling sont deux techniques utilisées pour optimiser les performances en contrôlant la fréquence à laquelle une fonction est exécutée. C'est particulièrement utile pour les événements qui peuvent se déclencher rapidement et de manière répétée, comme le redimensionnement de la fenêtre, le défilement ou la saisie de texte.

Debouncing

Le debouncing retarde l'exécution d'une fonction jusqu'à ce qu'un certain temps se soit écoulé sans qu'elle ait été appelée. C'est comme attendre que quelqu'un ait fini de parler avant de répondre.

Cela permet par exemple d'attendre que l'utilisateur ait fini de taper avant de lancer une requête de recherche.

Throttling

Le throttling garantit qu'une fonction n'est appelée qu'une seule fois au cours d'un intervalle de temps donné. C'est comme appuyer sur un bouton qui a un temps de rechargement. Cela est utile par exemple lorsque l'on veut faire du défilement infini : Charger plus de contenu au fur et à mesure que l'utilisateur fait défiler la page, mais pas à chaque pixel de défilement.

Pour en savoir plus

Pour une explication plus détaillée et des exemples de code, vous pouvez consulter le lien suivant : **<https://www.greatfrontend.com/questions/quiz/explain-the-concept-of-debouncing-and-throttling>**

Cross-Origin Resource Sharing

Requêtes HTTP

Requêtes *cross-domain* et sécurité

L'objet XHR doit respecter la politique de même origine (same-origin policy).

Cependant, la norme de 2012 prévoit que le navigateur puisse dans certaine circonstance relaxer cette règle rendant possible l'utilisation de XHR pour faire des requêtes cross-domain.

CORS (Cross-Origin Resource Sharing) est un mécanisme normalisé par le W3C qui permet de faire des requêtes cross-domain.

Le principe consiste en l'ajout dans l'entête de l'attribut **origin**. Le serveur répondra en incluant les attributs **access-control-allow-credential** et **access-control-allow-origin**.

Aller plus loin https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

▼ General

Request URL: https://www.googleapis.com/discovery/v1/apis/youtube/v3/rest
Request Method: GET
Status Code: 200 (from disk cache)
Remote Address: 172.217.18.202:443
Referrer Policy: no-referrer-when-downgrade

▼ Response Headers

access-control-allow-credentials: true
access-control-allow-origin: http://localhost:8080
access-control-expose-headers: Cache-Control,Content-Length,Content-Type,Date,ETag,Expires,Server,Vary,X-Go
kend-Request-Cost
age: 74
alt-svc: hq=":443"; ma=2592000; quic=51303431; quic=51303339; quic=51303338; quic=51303337; quic=51303335,
ma=2592000; v="41,39,38,37,35"
cache-control: public, max-age=300, must-revalidate, no-transform
content-encoding: gzip
content-length: 51128
content-type: application/json; charset=UTF-8
date: Fri, 26 Jan 2018 09:12:31 GMT
etag: "-iA1DTNe4s-I6JZXPt1t1Ypy8IU/IPcVNse7nmUQLqz58rb8Gkd1X1c"
expires: Fri, 26 Jan 2018 09:17:31 GMT
server: GSE
status: 304

```
vary: X-Origin  
vary: Origin  
x-content-type-options: nosniff  
x-frame-options: SAMEORIGIN  
x-xss-protection: 1; mode=block
```

▼ Request Headers

⚠ Provisional headers are shown

Origin: http://localhost:8080

Referer: http://localhost:8080/

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/63.0.ari/537.36

Exemple de requête HTTP *cross-domain*

Le cas particulier de JSONP

Avant 2012, un objet XHR ne permettait pas de faire de requêtes cross-domain (exception faite du passage par un proxy). Cela était en particulier dû à la politique de même origine.

Plusieurs solutions ont été proposées pour contourner le problème, en particulier JSONP pour JSON Padding, par Bob Ippolito (cf. <http://bob.ippoli.to/archives/2005/12/05/remote-json-jsonp/>).

Le principe repose sur l'ajout dans les paramètres d'un paramètre définissant le nom d'une fonction de callback (**monCallBack**, dans l'exemple ci-contre) à appeler lors de l'envoi de la réponse par le serveur.

L'intérêt de cette méthode c'est qu'elle demande peu d'investissement côté serveur. Par contre, en terme de sécurité, si le serveur contacté présente des vulnérabilités (injection Javascript par exemple) alors le serveur source est de fait aussi exposé.

Aller plus loin <https://www.ionos.fr/digitalguide/sites-internet/developpement-web/jsonp/>

Entête de la requête HTTP



Content-Length: 2094
Content-Type: application/json; charset=UTF-8
Date: Fri, 26 Jan 2018 09:32:38 GMT
Expires: Fri, 26 Jan 2018 09:37:38 GMT
Vary: Accept-Encoding
Via: 1.1 varnish-v4
X-Varnish: 400722221
▼ Request Headers view source
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
Connection: keep-alive
Cookie: l8-as01
Host: ws.meteofrance.com
Referer: http://localhost:8080/
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/63.0.3239.132 Safa
▼ Query String Parameters view source view URL encoded
callback: monCallback

La réponse obtenue

```
monCallback({"result":{"france":[{"indicatif":"130680","nom":"Paradou","type":"ville","idPa
```

Speaker notes